

When Updates Backfire: A Black-Box Security Analysis of Desktop Software Update Mechanisms

Jie Wan^{1,†}, Pengcheng Xia^{1,†}, Haoyu Wang^{1,‡}
¹*Huazhong University of Science and Technology**
 {jiewan, xpc357, haoyuwang}@hust.edu.cn

[†]*Equal contribution.* [‡]*Corresponding author.*

Abstract

Desktop software has become an essential component of modern computing, and software updates are the primary mechanism to patch vulnerabilities and deliver security fixes. However, the update process itself introduces new attack surfaces, particularly when validation of update data is incomplete or improperly enforced. Nevertheless, these risks in desktop updates are less studied because most update clients are closed-source and complex. We present **UpdSight**, a black-box framework that tests update security by simulating MitM attacks in realistic settings. **UpdSight** operates by emulating man-in-the-middle scenarios, intercepting traffic during the update process, and automatically validating the presence of critical weaknesses. By combining traffic interception, payload integrity inspection, and behavior monitoring, **UpdSight** provides a comprehensive assessment of update trust models across diverse software categories. We adopted **UpdSight** to 85 widely-used desktop applications. The results show 22 exploitable vulnerabilities, including downgrade, manifest manipulation, installer hijack, and path traversal. Among these, 16 have been confirmed by vendors. In addition, 5 CVE identifiers were assigned, covering vulnerabilities in 8 software products. Our findings highlight recurring design flaws, such as unsigned manifests and weak rollback checks, which allow attackers to gain code execution through the update channel.

1 Introduction

Software update mechanisms are a critical part of modern computing, powering timely patch delivery and feature deployment across desktop applications, mobile platforms, IoT devices, and industrial control systems. By automating the installation of security fixes, they play a central role in keeping systems safe and reliable. However, these same features

create a powerful attack vector: remote servers can trigger privileged code execution on end-user devices, often with minimal user oversight.

Several high-profile incidents demonstrate the severe consequences of compromised update processes. For example, the NotPetya malware exploited the update mechanism of the Ukrainian tax software MeDoc, where attackers compromised the update server to deliver a malicious payload signed with a stolen certificate, resulting in massive global damage [1]. Similarly, the ASUS Live Update utility was hijacked in the so-called ShadowHammer attack, where legitimate update channels were used to distribute backdoored software to thousands of devices [2]. In the SolarWinds supply chain attack, attackers inserted malicious code into signed updates of the Orion platform, affecting U.S. government agencies and Fortune 500 companies [3].

Recent studies have highlighted the security risks of software update mechanisms, primarily in Internet of Things (IoT) and automotive systems. Chowdhury et al. [4] analyze over-the-air updates in autonomous vehicles via a detailed threat model, identifying numerous vulnerabilities and proposing mitigation strategies. Wu et al. [5] introduce ChkUp, a system that detects firmware update flaws by analyzing verification logic, uncovering 25 new CVEs across diverse devices. Usman et al. [6] examine CVE trends in software and firmware updates, finding that these vulnerabilities are often more severe and frequently caused by signature or certificate validation failures. These works reveal recurring weaknesses such as missing authentication, improper certificate validation, and insufficient rollback protections. Yet, there is still no systematic black-box approach for assessing closed-source desktop software update logic in realistic network-adversary settings, where interception and manipulation can occur without compromising build systems or signing keys.

Challenges and Our Solution. In this paper, we build **UpdSight**, an automated black-box pipeline for detecting vulnerabilities in closed-source desktop software updates. Designing such a system revealed several key challenges that shaped our approach, which we outline below.

*The full name of the authors' affiliation is Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology.

(1) Challenge 1 — Opaque, fragmented update logic across GUIs and services. In many desktop applications, the update process is buried in proprietary binaries and may be distributed across background services, helper processes, and interactive UI components. These triggers are often activated only under specific runtime conditions, making them difficult to locate and execute in a black-box setting. To address this, **UpdSight** combines Update-Initiator and Domain-Tracer to reliably drive the native update workflow and capture all related network traffic. Scripted installation and update checks, combined with DNS redirection and controlled certificate setup, ensure that the complete update sequence is reproducible and fully observable for analysis.

(2) Challenge 2 - Stateful, multi-stage update pipelines with dispersed attack surface. Desktop update workflows typically involve several dependent stages such as metadata retrieval, target file selection, integrity verification, and installation. Weaknesses in any stage can enable manipulation of update content, but single-layer inspection tools cannot effectively validate exploitability across the full chain. **UpdSight** integrates MIMT-Insight to intercept and analyze update communications, Response-Crafter to pass malicious payloads that mimic legitimate files, and Exploit-Validator to confirm whether these payloads are executed or cause state changes. This approach enables the end-to-end testing of real attack scenarios, rather than relying solely on static checks.

Evaluation and Findings. We applied **UpdSight** to 85 widely-used desktop applications. Our analysis shows that many update clients rely on unsigned or weakly protected manifests, incomplete installer checks, and fragile archive extractors. As a result, we discovered 22 exploitable vulnerabilities, including downgrade, manifest manipulation, installer hijack, and path traversal. Among them, 16 have been confirmed by vendors and vulnerabilities affecting 8 applications have been assigned 5 CVE identifiers, highlighting that missing validation in update workflows still enables practical code execution attacks in real desktop software.

Summary. **UpdSight** makes the following key contributions:

- We provide a systematic taxonomy of desktop software update attack surfaces and define four security rules.
- We design and implement **UpdSight**, a black-box framework for automated detection of update vulnerabilities under MitM conditions.
- We evaluate 85 popular desktop applications and discover 22 exploitable vulnerabilities, 16 confirmed by vendors, with 5 CVE identifiers assigned to vulnerabilities affecting 8 applications.

We hope **UpdSight**¹ can serve as a foundation for detecting update-related vulnerabilities in desktop and other

software ecosystems and provide practical insights into the design of secure fetch-execute pipelines in complex applications.

2 Background

In this section, we provide background knowledge of desktop software update mechanisms and introduce the architectural components that make them vulnerable to security risks such as MitM attacks.

2.1 Desktop Software Update Architectures

We categorize software update mechanisms into two main types:

- **Monolithic updaters:** The software client includes a built-in updater that handles everything from metadata fetching to payload installation. These are common in standalone tools or games.
- **Modular updaters:** The software defers update logic to a separate helper process or update module, which can be reused across product lines. Examples include update agents for browsers or runtime platforms like Adobe or Java.

Most update mechanisms rely on HTTP or HTTPS as the underlying transport protocol. While HTTPS theoretically ensures confidentiality and integrity, many implementations suffer from improper certificate validation or fall back to plain HTTP. This leaves the communication channel vulnerable to interception and manipulation. Additionally, hostname resolution is typically performed via DNS, which introduces additional risks such as DNS spoofing.

Both monolithic and modular architectures are therefore subject to a common set of vulnerabilities—primarily stemming from insufficient verification of data retrieved over untrusted networks. This paper focuses on detecting such flaws in real-world software clients using a black-box MitM simulation framework. Our goal is to identify violations in certificate validation, data integrity, and update logic by observing actual software behavior under adversarial conditions.

2.2 Desktop Software Update Workflow

As illustrated in Figure 1, a typical software update process involves a multi-phase interaction between the client-side updater module and a set of remote services, including DNS, configuration servers, and file delivery infrastructure such as CDNs. This subsection reconstructs the update workflow in detail, with emphasis on the trust boundaries and potential attack surfaces exposed at each phase.

¹Our artifacts are available at <https://doi.org/10.5281/zenodo.17982402>.

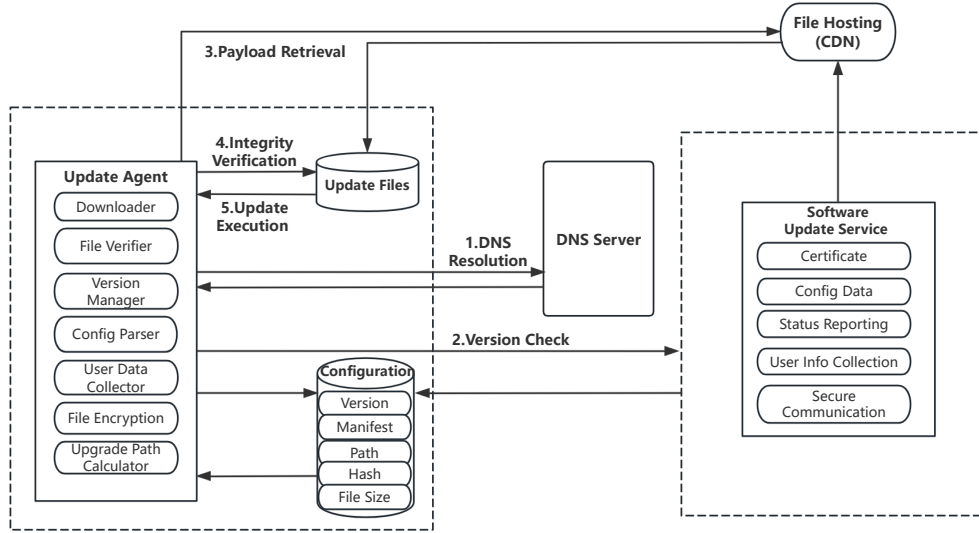


Figure 1: A common desktop software update workflow.

P1: DNS Resolution. The update procedure is typically initiated by the updater, which may run inside the main application or as a background service. It first performs a DNS query to locate the update server. Since most clients rely on system-configured DNS resolvers, an attacker who can spoof or poison DNS responses may redirect the updater to a malicious host. This makes DNS the first opportunity for traffic redirection in the update workflow, and later phases must rely on certificate validation or metadata checks to prevent compromise.

P2: Version Check. After locating the server, the updater downloads a manifest describing the latest version, file list, and hashes. Because this manifest dictates which payloads will be fetched, it is a sensitive target. If attackers can forge or tamper with it, they can trigger downgrade attacks, redirect payload URLs, or insert malicious entries. Without strong authentication, the manifest becomes an easy point of compromise that undermines later verification.

P3: Payload Retrieval. After obtaining the manifest, the updater downloads the update binaries from vendor servers or CDNs. Although HTTPS is widely used, many implementations fall back to plain HTTP, disable certificate checks, or embed hard-coded keys. These practices leave the channel vulnerable to man-in-the-middle tampering. If transport security fails, an attacker can inject arbitrary executables at this stage, making the download process a direct entry point for code substitution.

P4: Integrity Verification. Once payloads are retrieved, the updater should validate their authenticity before installation. Secure implementations check a digital signature bound to the payload, but many systems rely only on weak checksums or trust values contained in an unsigned manifest. Such incomplete checks allow tampered files to be accepted as

valid. If verification logic is weak or bypassed, the entire security of the update workflow collapses at this point.

P5: Update Execution. After verification, the updater installs or executes the new binaries, often with elevated privileges. At this stage, any earlier compromise directly translates into arbitrary code execution on the host. Even when verification exists, logic flaws such as accepting rollbacks or incomplete checks can let malicious payloads through. Because the execution step runs with high trust, it amplifies the impact of any weakness in the preceding phases.

2.3 Security Considerations in Software Update Implementations

In this section, we first define a reasonable threat model based on the software update process mentioned above, and finally summarize the 4 security rules that must be followed for software updates.

Threat Model and System Assumptions. We consider remote adversaries who interfere with the delivery or verification of software updates, excluding attacks that require prior local access, such as privilege escalation or filesystem manipulation. We focus on vulnerabilities that are triggerable during the update process itself, including attacks that exploit incomplete or improper validation of update-related data across different update stages, as detailed in §3. We assume that the vendor’s signing keys, build environment, and update servers are uncompromised. The update client is trusted to implement verification logic correctly, while its processing of remotely supplied data constitutes the main attack surface. The network is fully untrusted, and transport-layer protections such as TLS are not assumed to guarantee integrity or authenticity. Under these assumptions, the adversary can intercept,

Table 1: Security rules for desktop software updates.

No.	Security Rules	Protected Phases
R1	Secure Transport	P1, P3
R2	Metadata Integrity	P2
R3	Payload Authenticity	P3, P4
R4	Path Integrity	P5

modify, inject, or redirect update traffic, but cannot forge digital signatures, compromise vendor infrastructure, or obtain prior access to the target host.

Based on the software update process and the threat model, we have summarized four security rules that must be followed during the software update process, as summarized in Table 1.

R1: Secure Transport. Update clients must communicate with servers over authenticated and encrypted channels. If transport security is absent or certificate checks are disabled, an adversary can eavesdrop on or modify traffic during both name resolution and content retrieval. This rule is especially critical for P1 and P3, where tampered responses or substituted binaries could otherwise be accepted as legitimate.

R2: Metadata Integrity. The manifest specifies which version to install, where to fetch payloads, and how to verify them. Without cryptographic validation, attackers can forge metadata to redirect downloads, trigger downgrade attacks, or insert malicious entries. This rule primarily protects P2, where the absence of integrity guarantees directly undermines later checks.

R3: Payload Authenticity. Downloaded binaries must be validated before execution. Without authenticity checks, attackers who control the channel can replace executables, and the updater will install them with high privileges. Authenticity must also cover freshness: even a legitimately signed but outdated version should be rejected, otherwise an adversary can mount rollback attacks by replaying older packages. This rule is essential for P3 and P4, ensuring that even if network traffic is compromised, only genuine and current binaries are installed.

R4: Path Integrity. The installation process must confine updated files to their intended directories and prevent overwrite of arbitrary system resources. Without strict path integrity, attackers can craft archive entries or file paths to escape the update directory. This rule mainly applies to P5, where improper path handling can escalate a network injection into a full system compromise.

These four rules jointly define the baseline security requirements for software updates under our threat model. While each rule primarily protects different phases of the workflow, they are not entirely independent. For instance, payload authenticity cannot be guaranteed if metadata integrity is broken, and secure transport is necessary to make later checks meaningful. The update procedure resembles a chain: the compromise of any single link can allow adversaries to escalate their

control. In the next section, we show how concrete attacks emerge from violations of these rules.

3 Attack Vectors of Software Updates

We next describe concrete attack vectors that arise when the security rules are violated. Each vector shows how a missing check or weak defense in the update workflow can be exploited, such as modifying metadata, injecting outdated payloads, or bypassing transport protections. These cases are not isolated: they appear repeatedly across different software and systems, highlighting common mistakes in update design. In the following, we present representative attack types (AT1 - AT5) and analyze their impact.

3.1 AT1: Information Disclosure

If the software violates R1, it may cause information leakage, enabling attackers to capture users’ private data. This typically occurs during P2 or subsequent runtime operations. In such an attack, the adversary positions a man-in-the-middle proxy that transparently relays traffic to the update server while secretly collecting sensitive data. Users are unaware of the interception and their personal information is exposed. For example, when the update client communicates in plaintext or fails to validate the server’s certificate, login requests containing account information such as phone numbers or email addresses can be intercepted. Once obtained, these credentials may be reused by the attacker and even allow unauthorized logins from remote locations.

A representative example is CVE-2024-7570, in Ivanti ITSM, where improper certificate validation allowed a MitM attacker to forge tokens and impersonate any user. In practice, the attacker could intercept login traffic, obtain valid session information, and then reuse it to access the management system with full privileges.

3.2 AT2: Downgrade Attack

Violating R1 and R3 during P4 can lead to downgrade attacks. In this scenario, although the installer’s signature is verified, the update client fails to check the version of the received package. A MitM adversary can tamper with update traffic by intercepting the installer request and substituting a lower-version package. As a result, the client installs an outdated release that reintroduces known vulnerabilities and expands the attacker’s options for exploitation. Unlike information-leakage attacks, the adversary does not need to forward all traffic; instead, only installer download requests are filtered and replaced.

A representative case is Windows Downdate [7]. The attack exploits two key vulnerabilities, CVE-2024-21302 and CVE-2024-38202, targeting the Windows Update stack and allowing privilege escalation by modifying system restore

points to execute downgrades. The attacks succeeded because version checks were missing in the update workflow, showing how downgrade paths can be exploited even when signature verification is in place.

3.3 AT3: Path Traversal Attack

If the software violates R1 and R4 during P5, there is a path traversal attack, which extracts the file to the wrong directory and then uses the DLL to hijack or directly overwrite the executable file on the user's system to indirectly lead to code execution. Many update systems deliver installers as compressed archives and rely on custom extraction routines. When these routines fail to enforce directory constraints, malicious payloads embedded in the archive can escape the intended path. An on-path network attacker only needs to intercept the request for the update package and replace it with a crafted archive. During extraction, files are written to arbitrary system paths, allowing the adversary to overwrite executables or plant malicious DLLs that hijack normal execution.

One of the most well-known manifestations of this vulnerability is the Zip Slip attack, where improperly validated file paths inside archives allow overwrite of executables or configuration files outside the target directory. Such files may later be executed by the system or invoked during normal user operations, leading to arbitrary code execution. Even if direct code execution is not achieved, overwriting configuration or sensitive data can severely compromise system integrity. The root cause is the lack of strict path validation in decompression routines, which turns a routine unpacking step into an opportunity for privilege escalation.

3.4 AT4: Installer Hijack Attack

Violating R1 and R3 during P3 creates the risk of an installer hijack attack. In this attack, the update client requests an installer package but does not strictly verify its authenticity. The attacker can intercept the download request, replace the installer with a crafted executable, and still forward the rest of the traffic normally. Because the client executes the installer with elevated privileges, the malicious binary gains full control of the system once it is launched.

As illustrated in Figure 2, installer hijacking occurs when the update client is redirected to download an executable from an attacker-controlled server. In the case of CVE-2016-10629, affecting the Windows-based nw-with-arm installer for NW.js, the update process fetched binary resources over plain HTTP without authentication or cryptographic protection. A man-in-the-middle adversary could tamper with DNS responses, redirect the client to a spoofed host, and replace the installer with a malicious executable. Because the application failed to validate the server certificate or perform signature/hash

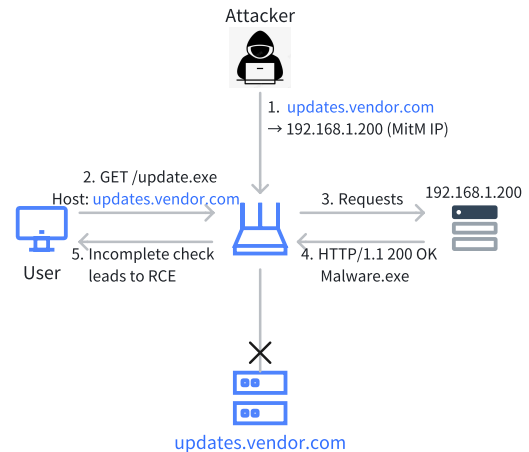


Figure 2: The installer hijack attack of CVE-2016-10629.

verification on the downloaded file, the attacker-supplied binary was executed during installation with standard privileges, resulting in remote code execution.

3.5 AT5: Manifest Manipulation Attack

Violating R1 and R2 during P2 enables attackers to tamper with update manifests. These files (often JSON or XML) contain critical metadata such as version numbers, file hashes, and download URLs. When manifests are transmitted without authentication or cryptographic protection, the adversary can intercept the response and alter its contents. By injecting a malicious URL or replacing a checksum with one computed over a trojanized binary, the attacker ensures that later integrity checks pass while the client actually fetches and installs a malicious payload.

A pertinent example is CVE-2021-33879, which affects Tencent GameLoop. The vulnerability stems from the use of insecure HTTP connections for update downloads. A man-in-the-middle attacker could spoof the XML document used to describe the update package, replacing legitimate URLs with malicious ones that point to arbitrary Windows executables. Since the only integrity verification was a basic MD5 checksum comparison, and MD5 checksum was embedded within the XML and can be easily forged, the update client would accept the tampered package and proceed to execute the malicious binary on the victim's machine, enabling full remote code execution.

4 UpdSight: A Semi-automated Framework for Desktop Software Update Security

Design Rationale. To systematically evaluate whether real-world software violates the four security rules, we design

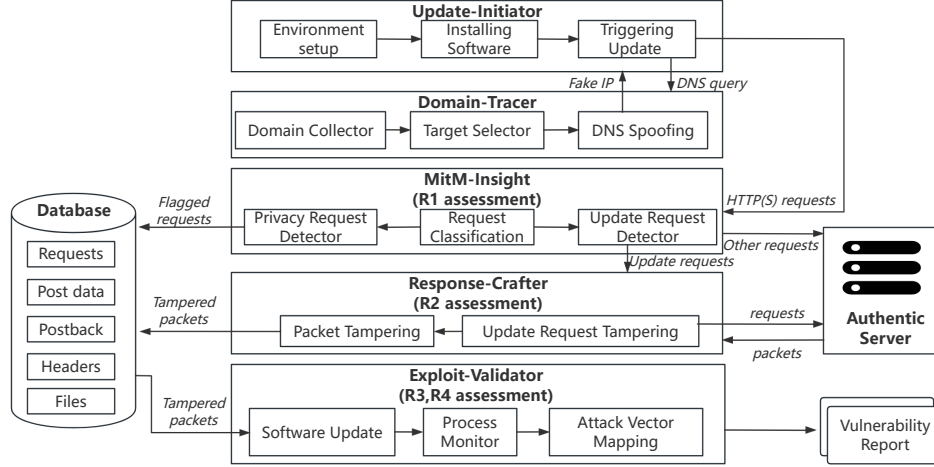


Figure 3: The architecture of UpdSight.

UpdSight to emulate an on-path adversary and observe how each defense step responds. The system intercepts update traffic, injects crafted payloads, and monitors execution effects on the client, creating a realistic adversarial environment. As discussed in §2.3, R1 is the prerequisite since no tampering is possible without transport compromise, while R2, R3, and R4 independently protect metadata integrity, payload authenticity, and path handling. By combining these checks, **UpdSight** determines which rule violations occur together and maps them to feasible attack vectors, ensuring that only practically exploitable chains are reported.

Overview of UpdSight. Figure 3 shows the overall architecture of **UpdSight**. Among the five modules, *Update-Initiator* automates installation and reliably triggers the native update workflow; *Domain-Tracer* simulates DNS hijacking and redirect traffic; *MitM-Insight* intercepts and classifies requests to test secure channel enforcement (R1); *Response-Crafter* forges update metadata and payloads to probe manifest manipulation and downgrade logic (R2); and *Exploit-Validator* monitors the client system to confirm code execution or path tampering, revealing violations of installer and deployment integrity (R3 and R4). Together, these components enable end-to-end black-box analysis, from redirecting traffic to validating exploit success. Note that R1, R3, and R4 can be tested automatically, while R2 often requires manual inspection due to the complex format of metadata, making the workflow semi-automated. In the following subsections, we detail the role of each module and how they jointly realize the semi-automated detection workflow.

4.1 Update-Initiator: Adversarial Environment Setup

Update-Initiator prepares a controlled MitM environment and drives the target’s native update routine in a repeatable way.

It has two steps: (1) environment preparation, which enforces an adversary-controlled proxy so that all update traffic is routed through us; and (2) update triggering, which invokes the program’s native updater and steers it to completion.

Environment setup. We boot a fresh VM snapshot and configure the system so that all outbound traffic traverses a host-side interception stack, which is later used by *Domain-Tracer* (§4.2) and *MitM-Insight* (§4.3). The module applies system proxy settings, and preloads DNS redirection rules. As a result, installation time traffic such as first run bootstraps, dependency fetches, and self-updates becomes observable and controllable. With the interception path in place, the module installs the target via mainstream package managers (e.g., Chocolatey, Scoop, Winget) or standalone installers (.exe, .msi). It records installation side effects, including registry entries, scheduled tasks, services, and file system artifacts, which later guide the update trigger.

Update triggering. The main task of Update-Initiator is to learn how the program starts its updater, record the first effective trigger, and keep the procedure reproducible. It performs an initial discovery pass that tests three paths in order: it launches candidate binaries from the installation directory while skipping uninstallers and helper tools and using documented flags such as /update when available; it starts matching scheduled tasks identified by path, author, or working directory; and it starts newly created services while logging their account, privileges, and spawned processes. The first path that yields a version check, a metadata fetch, or a payload download is saved as the trigger, and responses are passed through and cached to form a baseline. The VM is then restored to a clean snapshot and the recorded trigger is replayed to drive repeatable tests of the same update path; later runs may deliver edited manifests or forged payloads (see §4.4). After each run, the VM is reset to the clean snapshot.

Table 2: Allowlist categories used by Domain-Tracer. Domains outside the allowlist are redirected.

Category	Examples	Rationale
Trust chain (CRL/OCSP)	ctldl.windowsupdate.com	keep TLS validation intact
Package managers	chocolatey.org, scoop.sh	enable automated installation
Generic telemetry/service	watson.events.data.microsoft.com	reduce noise, not update-carrying
Non-update CDNs	store-images.s-microsoft.com	serve images or static files, not update binaries

By combining these strategies, Update-Initiator completes the initial construction of the adversarial environment and automates both application deployment and the activation of diverse update mechanisms.

4.2 Domain-Tracer: DNS Hijack Simulation

Domain-Tracer emulates DNS hijacking in Phase P1 and steers only update-scoped traffic to *MitM-Insight* (§4.3). By scoping interception with a domain allowlist, it keeps benign or bootstrapping domains out of the attack path, which preserves system stability and yields a realistic attacker model during installation and first run.

The domain allowlist. Before any redirection, Domain-Tracer applies a host-matching rule against an allowlist of domains that are needed for OS trust and installation but are unrelated to the target’s update logic. Concretely, we include: (1) trust-chain infrastructure such as CRL/OCSP endpoints to avoid breaking TLS validation, (2) package-manager infrastructure used by *Update-Initiator* to deploy the target, (3) generic telemetry or service domains that only provide background data collection or UI content, and (4) non-update CDNs that host static resources like images or scripts. Vendor update hosts and their associated CDNs (i.e., those serving version manifests, update binaries, or installers) are excluded by default. During installation, newly discovered prerequisites may be added only if they do not serve version checks, manifests, or installers. Table 2 summarizes categories and rationale.

Routing policy. Domain-Tracer runs as a lightweight HTTP/HTTPS forward proxy. For each outbound connection, it reads the target host and port from the HTTP `Host` header or the TLS SNI, then evaluates rules in a fixed priority order: (1) exact host match; (2) suffix or pattern match; (3) otherwise apply the default action. If the domain is on the allowlist, the request is forwarded unchanged; otherwise, the flow is redirected to *MitM-Insight* for observation and possible modification as part of the R1 evaluation.

4.3 MitM-Insight: Traffic Interception and Classification

MitM-Insight is responsible for detecting whether a software update violates R1. It is a programmable HTTPS endpoint that pretends to be the vendor update server during tests. Traffic

is first redirected by Domain-Tracer (§4.2). MitM-Insight listens on ports 80 and 443 with domain-matched certificates, builds a canonical log for each request, and fetches a baseline response from the real server. It can return the baseline as is or hand the response path to Response-Crafter (§4.4) for substitution. Figure 4 shows this flow from TLS handshake to response delivery.

Request interception. MitM-Insight accepts client TLS handshakes using certificates minted for the target domains. This lets it terminate HTTPS without user warnings and observe the full request. For each connection, it records method, URL, headers, and a short summary of the body, and it keeps only the first copy of repeated requests to avoid noise. In parallel, it forwards the request to the official server to obtain a baseline, which is stored for later comparison or pass-through.

Traffic filtering and leakage detection (R1 assessment). After capture, MitM-Insight scans URLs and headers for update-related and sensitive terms such as `update`, `token`, `email`, and `device_id`. Matches are kept in a separate log with the request context. It also parses response headers and URLs to find integrity hints such as `sha256` or `md5` and marks the client behavior that follows, so later modules can check whether those hints are enforced. The module reports three outcomes for R1: whether the client accepted our certificates, whether any plaintext traffic appears, and whether sensitive data is sent during updates.

By combining forged TLS certificates, systematic interception, keyword-based request filtering, and privacy-oriented analysis, MitM-Insight builds a comprehensive profile of each client’s update trust model. It allows assessment of whether TLS validation is enforced, whether sensitive data leaks occur, and whether integrity metadata is correctly utilized during software updates, and determines whether the software violates R1.

4.4 Response-Crafter: Payload and Metadata Tampering

This module prepares adversarial update responses. It performs the R2 test in a semi-automatic manner (as described in §4): it edits manifest fields such as version, URLs, and integrity metadata, serves the edited file, and judges whether the client accepts or rejects the tampered metadata. Manual review is required only when MitM-Insight (§4.3) surfaces a request that appears to contain an update configuration file; in

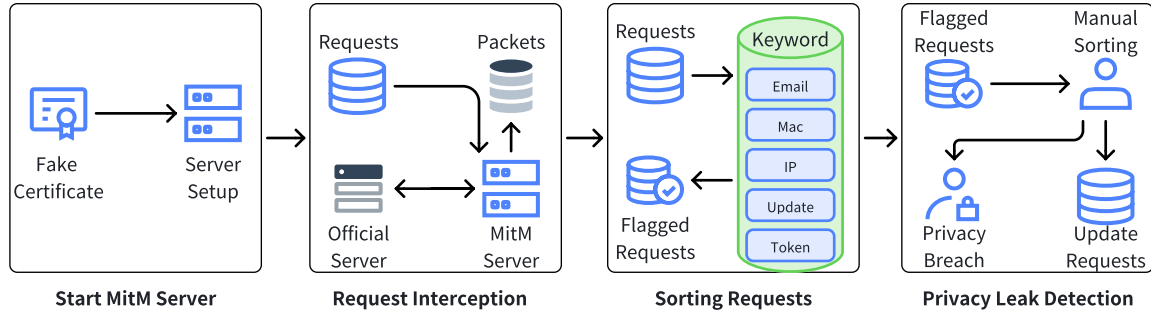


Figure 4: The workflow of MitM-Insight.

all other cases, the module proceeds fully automatically without operator input. It also supplies payloads that are used in Exploit-Validator (§4.5) later for payload tampering checks.

Payload replacement. On the first update attempt (described in §4.1), when the client requests a potentially executable resource (e.g., .exe, .msi, .zip, .tar), *Response-Crafter* classifies the file type using MIME and filename hints, fetches the original object from the vendor, records headers and a cryptographic hash, and caches the object to establish a baseline. On later attempts, it delivers controlled payloads along the same path: either a forged object from a local repository (such as an installer replacement or an archive prepared for path tests) or, to probe downgrade handling, an older package taken from the cache. In all cases, it preserves filenames and keeps response headers consistent (for example Content-Length, Content-Type, ETag) so that the client workflow remains unchanged.

Manifest modification (R2 assessment). Similar to payload replacement, the first manifest is delivered unchanged and cached while we record protection status (e.g., XML DSig) and key fields (version, URLs, checksums). On later attempts, the module returns an edited manifest. If the manifest is plaintext and unprotected, it applies an automatic template to lower the version, rewrite URLs to local payloads, and adjust or remove integrity fields. If the format is complex or protection is partial, an operator confirms the target fields and replacements. We mark R2 violated if the client accepts the edited manifest and proceeds with the update.

4.5 Exploit-Validator: Evidence Correlation and Attack Mapping

Exploit-Validator acts as the final consolidation point for all evidence collected during testing. From MitM-Insight (§4.3), it incorporates transport results for R1, such as acceptance of forged certificates or use of plaintext HTTP. From Response-Crafter (§4.4), it takes the R2 decision, which indicates whether the client accepted an edited manifest based on

Table 3: Mapping between attack types and violated rules.

Attack Vector	Violated Rules	Evidence (Exploit-Validator)
AT1	R1	Intercepted plaintext traffic
AT2	R1 + R3	Execution of outdated binary
AT3	R1 + R4	Files written outside target path
AT4	R1 + R3	Execution of forged patch
AT5	R1 + R2	Tampered manifest accepted

its subsequent requests. Exploit-Validator then monitors the host system to assess process execution and file placement, producing R3 and R4 results. Finally, it correlates all rule outcomes and maps them to specific attack types as defined in §3. Following this chain-of-proof model, the verdict reflects only those attacks that are practically exploitable under the assumed threat model.

Process monitor. At the system level, Exploit-Validator continuously observes how substituted payloads behave once delivered to the updater. For remote code execution, it inspects process creation events by comparing executable hashes, checking digital signatures, and recording parent-child relationships as well as exit codes. A match against the crafted payload confirms that a forged binary has been executed. For path-related exploits, it scans the file system for newly created or modified files, compares their actual location with expected installation directories, and detects permission or ownership changes. These checks yield definitive R3 and R4 results, which are then correlated with earlier network-side findings.

Attack vector mapping. Once all four rule results are available, Exploit-Validator applies the logical conditions defined in §3 to map them into attack vectors. As shown in Table 3, R1 is always a prerequisite, while R2, R3, and R4 represent parallel protections on metadata, payload, and path. A vector is confirmed only when rule violations are accompanied by concrete evidence on the client, such as execution of a forged

binary or files written outside expected directories. For example, insecure transport (R1) together with manifest tampering (R2) and observed payload execution corresponds to AT5, while insecure transport combined with path deviation during extraction (R4) leads to AT3.

5 Evaluation

5.1 Evaluation Setup

Dataset. Our evaluation initially considered a broad set of popular Windows desktop applications collected from reputable sources, including Chocolatey [8], GitHub Trending [9], the Microsoft Store [10], and official software websites. From this pool, we curated an initial candidate set of 120 applications by removing deprecated, legacy, or non-installable entries, retaining only actively maintained software that continues to receive regular vendor updates. The selection process considered both popularity and relevance, ensuring that the candidate set represents applications that users routinely rely on in practice. Among these 120 candidates, 85 applications could be successfully installed and exercised in our evaluation environment and were therefore selected for testing. The remaining applications could not be reliably evaluated due to practical constraints, which we discuss in §5.3. The resulting dataset spans a broad range of categories, including office productivity, communication and collaboration tools, browsers, media players, entertainment platforms, system utilities, and cloud services. All chosen applications are widely deployed, highly rated across multiple platforms, and cover typical desktop usage scenarios. The purpose of constructing such a dataset is to provide a realistic benchmark for assessing **UpdSight**’s ability to uncover undocumented vulnerabilities and insecure update practices at scale, under conditions that mirror the diversity and complexity of the real-world desktop ecosystem.

Evaluation Environment. To ensure consistency and prevent contamination across tests, all experiments were conducted in isolated virtual machines using VMware Workstation. The host system was a Windows 10 laptop equipped with an Intel Core i9 CPU (14 cores), 16 GB RAM, and a 500GB SSD. Each VM was set with 4 CPU cores, 16 GB RAM, and a 60 GB virtual disk. The operating system inside the VM was also Windows 10, and the VM was snapshotted before each test. The tool was implemented in Python with a modular design. Unlike other components, the **Update-Initiator** module requires administrator privileges, as it may trigger system services and scheduled tasks to initiate updates. In terms of execution environment, **Update-Initiator** and **Exploit-Validator** are deployed inside virtual machines, while **Domain-Tracer**, **MitM-Insight**, and **Response-Crafter** operate on the host machine.

For measurement, we considered two aspects. First, we assessed whether adversarial manipulation could lead to concrete exploitation, including remote code execution or unauthorized file placement. Second, we analyzed compliance with the four security rules (R1–R4), marking an application as vulnerable whenever a violation was observed. These criteria together allow us to quantify both exploitability and systemic weaknesses in real-world update mechanisms.

5.2 Degree of Automation

UpdSight is designed as a semi-automated evaluation framework. As is demonstrated in §4, the update triggering, traffic interception, and exploit validation stages are executed automatically. Manual effort is required only in two cases. First, for downgrade analysis (AT2), older installers must be retrieved manually when vendors do not provide public access to historical versions. Second, during metadata integrity assessment (R2), limited manual configuration is needed to inspect update configuration files and annotate key metadata fields, such as download URLs, version identifiers, file hashes, and sizes. All other stages of the evaluation pipeline operate without human input.

Quantitative breakdown. Across all 85 software samples, approximately 30% (25 samples) required manual review of their requests, while the remaining 70% (60 samples) were processed entirely automatically. In total, the manual effort amounted to about one hour, which was mostly triage (95%) and only minor annotation (5%). This represented less than 5% of the total campaign time, compared to the automated pipeline, which ran for approximately 20 hours.

An illustrative example. Figure 5 illustrates a representative example of how **UpdSight** detects a manifest manipulation vulnerability (AT5) by exercising the update workflow and observing rule violations. As shown in the figure, **Update-Initiator** first triggers the native update procedure, causing the client to resolve the vendor domain and issue a request for the update manifest. **Domain-Tracer** intercepts the DNS resolution and redirects subsequent update traffic to **MitM-Insight**, which identifies and extracts the upgrade check request. At this stage, corresponding to the R2 assessment point in the update pipeline, **Response-Crafter** performs the only manual step in the analysis: it replaces the intercepted manifest with a modified version that rewrites the update URL to an attacker-controlled binary and forges the associated integrity fields (e.g., MD5, SHA-256, and size). If the client accepts the modified manifest and proceeds with payload download and execution, **Exploit-Validator** observes the resulting behavior and confirms the violation by detecting the execution of the injected binary. This outcome indicates that the update mechanism fails to enforce metadata integrity, making it vulnerable to manifest manipulation attacks.

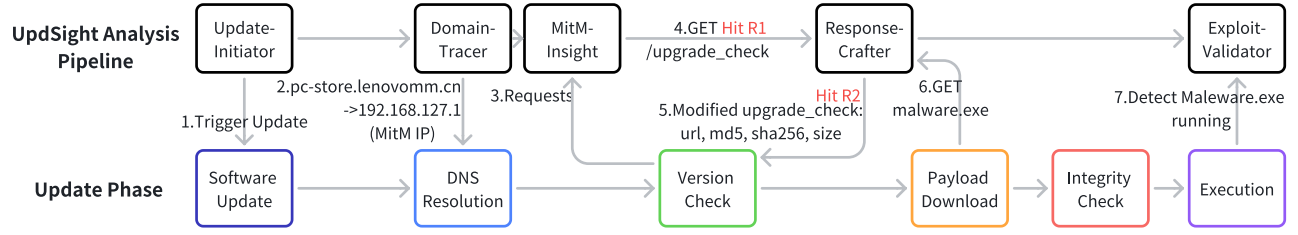


Figure 5: Detecting a manifest manipulation vulnerability (AT5) in the software update pipeline.

Table 4: Real-world vulnerabilities detected by UpdSight. (“○” indicates security rule violation, while “●” indicates security rule compliance).

No.	App Name	Downloads	Security Rule				Attacks (Vulnerabilities)	Reported	Confirmed
			R1	R2	R3	R4			
1	MSSoftOfficePLUS v3.10.0.28331	600M	○	●	●	●	AT1	✓	✓ (fixed)
2	Lenovo Store v9.0.1830.0926	20M	○	○	●	○	AT3, AT5	✓	✓ (CVE-2025-10495, fixed)
3	Lenovo PC Manager v5.1.120.6304	20M	○	○	●	○	AT3	✓	✓ (CVE-2025-10495, fixed)
4	Lenovo Browser v9.0.6.7021	20M	○	○	●	○	AT3, AT5	✓	✓ (CVE-2025-10495, fixed)
5	Lenovo Legion Zone v2.0.6.3131	20M	○	○	●	●	AT5	✓	✓ (CVE-2025-10495, fixed)
6	Lenovo Wallpaper Client v3.0.70.3301	20M	○	○	●	●	AT5	✓	✓ (CVE-2025-9319)
7	Lenovo LeCloud v2.501.25.0	20M	○	●	●	●	AT1	✓	✓ (CVE-2025-10699)
8	Internet Download Manager v6.42.41.1	5M+	○	○	○	●	AT2	✓	✓ (CVE-2025-56231)
9	GOG Galaxy v2.0.0.2	1M+	○	●	○	●	AT4	✓	✓ (CVE-2025-56232)
10	An Enterprise Collaboration Platform	1.2B	○	●	●	○	AT3	✓	✓ (fixed)
11	An Enterprise Cloud Storage Suite	20M	○	●	●	●	AT1	✓	✓ (fixed)
12	A Video Platform	50M+	○	○	●	●	AT5	✓	✓
13	A Software Store	20M+	○	○	●	●	AT5	✓	✓
14	An Online Document Service	20M+	○	○	●	●	AT5	✓	✓
15	An Office Suite	100M+	○	○	●	○	AT3, AT5	✓	
16	A Media Player	100M+	○	●	○	●	AT4	✓	
17	A Video Platform	20M+	○	○	●	●	AT5	✓	
18	A Messaging Client	5M+	○	○	●	○	AT3, AT5	✓	

5.3 Results of Individual Modules

We then analyze the experimental outcomes of each module in the UpdSight pipeline, focusing on their ability to expose violations of the four security rules. By presenting per-module findings, we show how individual stages contribute to the final detection results before reporting the overall evaluation.

Update-Initiator. In our experiments, 85 out of 120 applications produced update-related network requests such as version queries, manifest retrieval, or executable downloads, showing that their update routines can be automatically triggered in a controlled environment of Update-Initiator. The remaining 35 applications failed to exhibit update behavior and can be grouped into three categories. 14 applications were offline or minimally networked, requiring users to manually fetch installation packages from official websites. 12 applications were login-dependent, where update checks only occur after user authentication, preventing automated triggering without valid credentials. The last 9 applications were

essentially web-based services that did not rely on conventional update mechanisms and therefore exposed no observable update traffic. For these cases, we recorded the absence of update requests and excluded them from subsequent vulnerability validation, as their workflows cannot be exercised within our framework.

Domain-Tracer. In practice, Domain-Tracer redirected update traffic to MitM-Insight without disturbing the target applications. Across the entire dataset, all programs continued to install and launch normally, and no crashes or broken workflows were observed, showing that redirection was achieved transparently. During installation and first run, most applications reached out to a mix of vendor domains, trust-chain endpoints, and system services. On average, approximately 3 system-level requests per second, such as those to `tlu.dl.delivery.mp.microsoft.com`, were identified as unrelated to the application’s update logic and excluded from interception. After filtering out these background requests, a total of 625 requests, including vendor hosts and associated

CDNs, were redirected to MitM-Insight for interception and further analysis.

MitM-Insight. MitM-Insight intercepted a total of 625 unique requests from 85 applications during our experiments. Among these, 136 were recognized as update or privacy related, forming the primary input for subsequent analysis. Manual validation revealed a small number of misclassifications. In 18 cases, requests were flagged because they carried tokens such as update, yet the traffic itself was unrelated or malformed. Another 4 requests were marked as sensitive due to the presence of parameters resembling user identifiers, but their values turned out to be empty or nonsensical character sequences. Conversely, 12 requests that did not contain obvious indicators were overlooked, even though their responses later proved to carry updated metadata or private information. Although some misclassifications occurred, most update-related flows were still correctly separated from background noise and passed on for further testing.

Response-Crafter. Response-Crafter crafted responses for several attack categories, including manifest manipulation, path traversal, installer hijack, and downgrade attempts. The forged objects conformed to the expected format of update files and were processed by the clients along their normal update routines. In some cases this meant the payload was later rejected during signature checks or other validations, but the workflow itself proceeded without crashes or unexpected failures. These crafted inputs were then passed to Exploit-Validator, which determined in subsequent steps whether they resulted in successful exploitation.

Exploit-Validator. Exploit-Validator identified 22 vulnerabilities across the 85 tested applications. For the request-driven attacks (AT1/AT2), detection yielded no false positives or false negatives, as these cases are determined directly by content filtering and are further confirmed during R2 manual inspection. For the payload-driven attacks (AT3/AT4), detection produced zero false positives because each case is tied to a distinct hash or file-level trigger. We observed one false negative in a DLL sideloading case: the crafted update successfully delivered the malicious DLL, but it was loaded only later by a trusted binary, outside our monitoring window.

5.4 Overall Evaluation Results

Having examined the outcomes of each module in isolation, we now turn to the overall effectiveness of **UpdSight** when executing the full pipeline. As shown in Table 4, our large-scale, attack-based evaluation uncovered 22 previously undocumented vulnerabilities in 18 desktop software. 16 vulnerabilities have been acknowledged by vendors, 9 vulnerabilities have been fixed, while vulnerabilities affecting 8 applications have been assigned 5 CVE identifiers (with 4 applications in the same family sharing a single identifier), and the remainder are under coordination. Notably, 3 of these vulnerabilities were patched within a month of our report due to their severity,

underscoring both the impact of the flaws and the effectiveness of rapid vendor response.

Attack vectors. Among the 22 vulnerabilities, information disclosure (AT1) accounted for 3 cases, while manifest manipulation attacks (AT5) were the most prevalent, with 10 cases, where attackers were able to poison JSON or XML configuration files fetched over insecure channels. 1 case involved downgrade attack (AT2), and 2 cases involved installer hijack attack (AT4), allowing forged .exe or .dll files to be downloaded and executed. 6 cases were related to path traversal attack (AT3), enabling path traversal or unauthorized file overwrites during extraction. It is worth noting that some of these attack types overlapped within the same update process of a single application.

Violation of security rules. In addition to categorizing attacks, we analyzed violations of the four core security rules. 22 vulnerabilities reflected a failure in secure transport (R1), where update clients fetched critical metadata or payloads over unauthenticated HTTP channels or accepted spoofed certificates, exposing them to interception. Another 10 cases broke metadata integrity (R2), where manifests describing update content could be tampered with due to the absence of cryptographic protection. These manipulated manifests enabled the substitution or redirection of update packages. Violations of path integrity (R4) occurred in 6 cases, where unsafe decompression or unverified archive extraction enabled attackers to overwrite arbitrary files on the system using crafted archive entries. Payload authenticity (R3) was violated in 4 instances, where a lack of signature checks or weak version enforcement allowed malicious installers or outdated packages to be accepted and executed.

Vulnerable software. The vulnerabilities were not evenly distributed across application categories. Development and technical tools, along with entertainment software, showed greater exposure to the Manifest Manipulation Attack (AT5). Meanwhile, path traversal issues were more frequently observed in development tools and communication applications due to unsafe extraction logic. In contrast, many development tools enforced both signature checks and secure communications, resulting in fewer observed vulnerabilities overall. Additionally, we found that when one product from a development team exhibited security issues, other applications developed by the same team were more likely to suffer from similar vulnerabilities, suggesting reuse of insecure updater components or shared codebases.

5.5 Case Studies

Beyond the manifest manipulation attack (AT5) discussed in Figure 5, we present two real-world case studies to demonstrate how **UpdSight** can uncover complex, exploitable vulnerability chains within software update mechanisms.

Case study 1: archive injection via path traversal attack. (AT3) In this case, the application queried its update

server through an HTTPS POST request to check for the latest available version. The update logic only proceeded when the server reported a higher version number, but the client did not authenticate the response or verify the server certificate. Using **UpdSight**, we intercepted the reply and raised the version field so that the client immediately initiated an update. The updater then downloaded a compressed archive, which we replaced in transit with a maliciously crafted .7z file containing path traversal entries. The client extracted this archive without validating its structure or enforcing directory boundaries, which caused attacker-controlled files to be written into arbitrary system locations outside the intended update directory. As a result, the attack enabled the overwrite of sensitive user data, the planting of persistent executables, and ultimately indirectly executed malicious code. The vulnerability was rooted in a complete lack of authenticity checks on remote files, the absence of certificate validation, and an unsafe extraction routine that blindly trusted archive content. This chain enables silent exploitation, with the potential to overwrite system or user files, plant malicious binaries, or gain persistence. The severity of the vulnerability is high due to its stealth and exploitability.

Case Study 2: Downgrade Attack via configuration manipulation. In this case, the updater periodically fetched a configuration file over HTTPS that specified both the latest version number and the corresponding download URL. The client performed only a superficial signature validation of the downloaded binary, but it neither checked the freshness of the update nor enforced any protection against rollback. With **UpdSight**, we intercepted the configuration file in transit and modified the version field so that the client believed a new release was available. When the updater subsequently downloaded the payload, we substituted it with an older but legitimately signed version of the same software. Because the signature was valid, the client accepted the binary and installed it, effectively rolling the application back to a vulnerable state. This attack demonstrates how incomplete validation logic—specifically, missing certificate checks, unauthenticated configuration data, and the absence of version freshness enforcement—can be exploited to mount downgrade attacks. The exploit required no user involvement and immediately exposed the system to previously patched vulnerabilities, highlighting the risks of treating signed but outdated binaries as trusted updates.

6 Lessons Learned and Mitigation

We discussed the lessons learned from the desktop software security analysis and possible mitigations in the section.

Lessons learned. Our study yields the following lessons about the systemic weaknesses in desktop software update mechanisms:

First, transport checks remain the first point of failure. Many applications still skip TLS certificate validation or fall

back to plaintext HTTP. Once the channel can be intercepted, all later protections lose their value. This shows that without strict transport enforcement, the update pipeline is open to trivial attacks.

Second, update metadata is a critical but fragile trust anchor. The manifest often decides which files to download and how to check them. In many cases it is unsigned or only protected by weak checksums. Such design lets attackers replace update files or force a rollback without being detected.

Third, payload validation is incomplete without freshness. Signature checks alone do not stop an attacker from replaying old but still valid binaries. Without freshness enforcement, downgrade attacks are easy to launch. This gap shows that authenticity and freshness must always be verified together.

Last, deployment routines remain vulnerable to unsafe file handling. Many update systems extract archives without checking directory boundaries. Malicious archives can then place files outside the update folder and overwrite system resources. These problems show that low-level details like path sanitization are also crucial for security.

These lessons underscore that software update security cannot be guaranteed by a single mechanism in isolation. Missing validation at any one stage of transport, metadata, payload, or deployment could render all other protections ineffective. A secure update pipeline must therefore be treated as a chain, where every link is reinforced by cryptographic guarantees and safe engineering practices.

Mitigation. Addressing these issues requires a defense-in-depth approach rather than ad-hoc fixes. Update clients must enforce strict TLS validation by default, removing insecure fallbacks. Metadata files should be cryptographically signed and bound to specific versions so that tampering is immediately detectable. Payload validation should combine authenticity with freshness guarantees to prevent rollback, and operating systems should encourage developers to reject outdated binaries even when signatures are valid. Finally, developers must adopt safe, well-tested extraction libraries that enforce path integrity and eliminate traversal risks, rather than relying on custom unpacking code.

7 Discussions

Generality. The vulnerabilities uncovered by **UpdSight** stem largely from weaknesses in network-side validation, such as missing certificate checks, unsigned manifests, and insufficient freshness guarantees. These flaws are not tied to a particular platform or vendor, but reflect fundamental design issues in the way update clients fetch and verify remote data. As a result, the methodology of **UpdSight** is general. With minor adjustments to the triggering and parsing modules, the framework can be applied to other ecosystems such as Linux package managers, mobile or terminal applications, and even IoT devices. This generality shows that the attack surface of software updates is consistent across environments, and

systematic validation of network trust boundaries is essential for any secure update pipeline.

Limitation. While **UpdSight** provides systematic coverage of desktop update mechanisms, it is not yet fully automated. Certain steps, such as interpreting complex manifest formats or confirming exploit feasibility, still require manual analysis to avoid false positives. In addition, update workflows are highly diverse across applications, and our framework cannot always recognize or trigger every possible update path in a black-box setting. As observed in our experiments, a fraction of applications either failed to exhibit update behavior or relied on login-dependent or web-based services beyond our scope. These limitations suggest that while **UpdSight** effectively captures common and exploitable flaws, future work can leverage more advanced automation techniques, including program analysis and large language models, to better understand complex workflows and further extend coverage.

Responsible Disclosure. All confirmed vulnerabilities were disclosed to affected vendors under responsible disclosure practices. To protect the ecosystem while still conveying the security lessons, we anonymize related application names in this paper and avoid releasing proof-of-concept exploits or vendor-specific details. Instead, we present representative vulnerability patterns and abstracted case studies that demonstrate how flaws can be exploited, without enabling direct replication against unfixed targets.

8 Related Work

In this section, we review prior research related to software update security, including studies on software update mechanisms and security analysis of software update and compare how **UpdSight** differs from other tools.

Software update mechanisms. Software update design has also been studied from the perspectives of safety, security, and resilience [11–13]. Mugarza et al. [11] highlight the intertwined challenges of safety and security in mixed-criticality embedded systems, using OTA software updates as a key example. They propose a combined risk assessment in an automotive case study and suggest mitigation measures to balance update needs with functional safety requirements. Han et al. [12] further analyze update design principles for cyber-physical systems, showing how safety standards and update frequency may conflict in practice. Bohara et al. [13] discuss cyber resilience in automotive environments and emphasize the role of timely security updates in meeting regulatory and safety requirements. Other works focus specifically on the security design of IoT and firmware updates [14–22]. Catuogno et al. [22] review the unique challenges of updating IoT and embedded devices, including resource constraints, large-scale deployment, and lack of standardized distribution platforms.

Security analysis of software update. One important research direction closely related to update security is software

supply chain security [23–31]. For example, based on observations from industry and government organizations, Enck et al. [25] find that constrained IoT devices require lightweight yet robust OTA update mechanisms, where security must combine cryptographic protection, efficient protocols, and resource-aware design. Reichert et al. [26] conduct a systematic literature review and show that, while attention to build infrastructures and third-party dependencies is growing, important gaps remain, including limited focus on supply chains involving cloud components, few proposals addressing software distribution, and little use of threat modeling frameworks for identifying and validating threats.

Another line of work has studied the security of firmware and IoT software updates in depth [4–6, 32–37]. For example, Tsang et al. [37] introduce FANDEMIC, a new class of firmware attacks that maliciously misconfigure power management integrated circuits (PMICs) in IoT devices, leading to effects such as sensor data corruption, battery failure, and accelerated device aging. Wu et al. [5] propose ChkUp, a system that detects firmware update flaws by analyzing verification logic, uncovering 25 new CVEs across diverse devices. Usman et al. [6] examine CVE trends in software and firmware updates, finding that these vulnerabilities are often severe and frequently caused by signature or certificate validation failures. Collectively, these works reveal recurring weaknesses such as missing authentication, improper certificate validation, and insufficient rollback protections. Yet, there is still no systematic black-box approach for assessing closed-source desktop software update logic in realistic network-adversary settings, where interception and manipulation can occur without compromising build systems or signing keys.

Comparison with existing tools. There are some existing MitM and traffic-manipulation frameworks, such as mitmproxy [38], Tapioca [39], Bettercap [40], and Ettercap [43]. They primarily operate at the transport layer, enabling interception, redirection, and modification of update traffic. As such, they can partially approximate the functionality of **UpdSight**’s Domain-Tracer and MitM-Insight components, e.g., by redirecting domains or manipulating network responses. However, they treat update-related traffic as generic network flows and do not provide first-class support for update semantics. Even when combined with auxiliary monitoring techniques, such as binary-patching proxies (e.g., BDFProxy [41]) and process monitoring (e.g., procmon [42]), the analysis remains largely coarse-grained, focusing on generic attack surfaces such as binary replacement. In particular, they cannot automatically distinguish whether intercepted requests correspond to privacy-related or update-related files without relying on manually crafted scripts. Moreover, achieving end-to-end update vulnerability detection requires additional capabilities that are not native to MitM frameworks, such as update-stage identification, correlating modified metadata with subsequent update behavior, and validating exploitability through

Table 5: Comparison of UpdSight with existing tools. (“●” indicates native support for detecting update-related vulnerabilities, while “○” indicates partial support achievable only through ad-hoc scripting or external instrumentation).

No.	Name	Native Support Level	Security Rule Coverage				Focus
			R1	R2	R3	R4	
1	UpdSight	Native (semi-automated)	●	●	●	●	Update-mechanism vulnerabilities
2	Mitmproxy [38]	Script-based (interception layer)	●	○	○	○	Transport-level interception
3	Tapioca [39]	Script-based (interception layer)	●	○	○	○	Transport-level interception
4	Bettercap [40]	Script-based (interception layer)	●	○	○	○	Transport-level interception
5	BDFproxy [41]	Script-based (interception layer)	●	○	○	○	Transport-level interception
6	Procmon [42] (with MitM tools)	Manual	●	○	●	○	Installer hijacking

execution- and file-system-level evidence. While these steps can be approximated through scripting and APIs on top of MitM tools, doing so typically requires substantial application-specific logic and manual interpretation.

Consequently, as summarized in Table 5, existing MitM-based tools are effective at identifying R1 violations, such as insecure transport or improper certificate validation, and can be extended with process monitoring to validate certain R3 scenarios by observing whether replaced payloads are executed. In contrast, support for other update security rules is not provided natively. Uncovering violations of R2 and systematically validating R3–R4 typically requires ad-hoc scripting, auxiliary instrumentation, and substantial manual interpretation. As a result, the main scalability limitation of MitM-based approaches lies not in runtime overhead, but in the human effort required to develop and maintain scripts and to correlate network-level observations with update behavior across diverse applications.

In contrast, **UpdSight** is purpose-built for analyzing software-update workflows and directly addresses these limitations and encapsulates this update-aware logic into a unified and reusable analysis pipeline, enabling scalable detection across diverse applications with minimal manual effort. **UpdSight** targets content-handling vulnerabilities—the dominant class in our study, where 19 of 22 issues stem from incorrect parsing or validation of manifests and payloads, of which 16 involve update-configuration and path-traversal flaws that fall outside the analysis scope of existing network-centric MitM tools. **UpdSight** automates most of the analysis pipeline, enabling systematic detection of all R1–R4 issue types. In practice, as shown in §5.2, the manual effort required accounts for less than 5% of the total campaign time. The automated analysis pipeline runs unattended for approximately 20 hours, allowing the study to scale without continuous human involvement. By adapting its environment setup and interception modules to platform-specific behaviors, **UpdSight** can be extended to macOS, Linux, or mobile ecosystems.

9 Conclusion

In this paper, we present the first systematic analysis of desktop software update security under realistic man-in-the-middle conditions. By applying our black-box framework **UpdSight** to 85 popular applications, we uncovered 22 exploitable vulnerabilities, 16 of which were confirmed and have been assigned 5 CVE identifiers. These findings expose recurring flaws—such as unsigned manifests and weak rollback checks—that continue to undermine the security of update channels. Our work highlights the need for stronger, standardized defenses and provides a foundation for building more trustworthy update mechanisms in practice.

Ethical Considerations

We consider the ethical implications of this work from the perspective of the key stakeholders involved.

Software vendors. All vulnerabilities discovered in this study were responsibly disclosed to the corresponding vendors and we have followed up with affected vendors to track the remediation progress of the reported issues. The paper primarily focuses on vulnerabilities that have already been patched. We also include a small number of cases that have been publicly disclosed (e.g., assigned CVE identifiers) but were not yet fully patched at the time of writing. For these cases, we do not describe any attack techniques, exploitation conditions, or root causes beyond what is already available in the corresponding CVE records or public advisories. As a result, our descriptions do not lower the bar for reproducing or exploiting these vulnerabilities beyond what is already publicly known. For vulnerabilities that were neither patched nor publicly disclosed, we anonymize the affected software and completely omit technical details.

End users. Our findings aim to improve the security of software update mechanisms that end users rely on in practice. By identifying and reporting update-related vulnerabilities, this work helps reduce the risk of malicious updates and strengthens the integrity and authenticity of update workflows.

Security researchers and potential misuse. We provide high-level methodological insights without releasing harmful

payloads, detailed exploitation steps, or automated exploit scripts. Our system is not designed as an out-of-the-box exploitation framework, and all experiments were conducted in isolated local virtual machines. We believe that the security benefits to stakeholders outweigh the limited dual-use risks.

Open Science

We make the executable of **UpdSight**, and source code, and documentation available to the research community at <https://doi.org/10.5281/zenodo.17982402>, so that others can reproduce our analysis and extend our findings. This includes the automation pipeline for triggering updates and the experimental workflow for traffic interception, payload manipulation, and vulnerability validation. At the same time, we withhold proof-of-concept exploits and vendor-specific details to prevent misuse, striking a balance between openness and user safety.

Acknowledgements

This work was supported in part by the National Natural Science Foundation of China (grant No.62572209) and the Hubei Provincial Key Research and Development Program (grant No.2025BAB057).

References

- [1] A. Greenberg, “The untold story of notpetya, the most devastating cyberattack in history,” *WIRED*, 2018. [Online]. Available: <https://www.wired.com/story/notpetya-cyberattack-ukraine-russia-code-crashed-the-world/>
- [2] K. Lab, “Operation shadowhammer: Targeted supply chain attack in asus live update utility,” 2019. [Online]. Available: <https://securelist.com/operation-shadowhammer/89992/>
- [3] P. Baker, “The solarwinds hack timeline: Who knew what, and when?” 2021. [Online]. Available: <https://www.csoonline.com/article/570537/the-solarwinds-hack-timeline-who-knew-what-and-when.html>
- [4] N. I. Chowdhury and R. Hasan, “How trustworthy are over-the-air (ota) updates for autonomous vehicles (av) to ensure public safety?: A threat model-based security analysis,” in *2024 IEEE World Forum on Public Safety Technology (WFPST)*. IEEE, 2024, pp. 87–92.
- [5] Y. Wu, J. Wang, Y. Wang, S. Zhai, Z. Li, Y. He, K. Sun, Q. Li, and N. Zhang, “Your firmware has arrived: A study of firmware update vulnerabilities,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 5627–5644.
- [6] A. B. Usman and M. Asplund, “Update at your own risk: Analysis and recommendations for update-related vulnerabilities,” in *IFIP International Conference on ICT Systems Security and Privacy Protection*. Springer, 2025, pp. 97–110.
- [7] CVE Details, “An update on windows down-date,” <https://www.safebreach.com/blog/update-on-windows-downdate-downgrade-attacks/>.
- [8] “Chocolatey – the package manager for windows,” <https://chocolatey.org/>, 2024.
- [9] “Github trending – today’s most popular repositories,” <https://github.com/trending>, 2024.
- [10] “Microsoft store – official app marketplace for windows,” <https://apps.microsoft.com/store>, 2024.
- [11] I. Mugarza, I. Yarza, I. Agirre, F. Lussiana, and S. Botta, “Safety and security concept for software updates on mixed-criticality systems,” in *2021 5th International Conference on System Reliability and Safety (ICSRS)*. IEEE, 2021, pp. 171–180.
- [12] C. Han, X. Liu, B. Liu, and Q. Zhang, “Design of embedded remote software update system based on fpga+arm,” *Journal of Interconnection Networks*, vol. 22, no. Supp02, p. 2143040, 2022.
- [13] R. Bohara, M. Ross, S. Rahlfs, and S. Ghatta, “Cyber security and software update management system for connected vehicles in compliance with unece wp. 29, r155 and r156,” in *Software Engineering 2023 Workshops*. Gesellschaft für Informatik eV, 2023, pp. 41–53.
- [14] P. Emami-Naeini, J. Dheenadhayalan, Y. Agarwal, and L. F. Cranor, “An informative security and privacy “nutrition” label for internet of things devices,” *IEEE Security & Privacy*, vol. 20, no. 2, pp. 31–39, 2021.
- [15] F. Herbold, A. Reindl, H. Meier, M. Niemetz, and S. Krämer, “Secure software updates: Challenges and solutions for embedded iot systems,” in *9th Prague Embedded Systems Workshop*, 2021, p. 5.
- [16] M. Aziz Al Kabir, W. Elmedany, and M. S. Sharif, “Securing iot devices against emerging security threats: Challenges and mitigation techniques,” *Journal of Cyber Security Technology*, vol. 7, no. 4, pp. 199–223, 2023.
- [17] S. El Jaouhari and E. Bouvet, “Secure firmware over-the-air updates for iot: Survey, challenges, and discussions,” *Internet of Things*, vol. 18, p. 100508, 2022.
- [18] A. Langiu, C. A. Boano, M. Schuß, and K. Römer, “Up-kit: An open-source, portable, and lightweight update framework for constrained iot devices,” in *2019 IEEE*

39th international conference on distributed computing systems (ICDCS). IEEE, 2019, pp. 2101–2112.

- [19] B. Moran, H. Tschofenig, D. Brown, and M. Meriac, “A firmware update architecture for internet of things,” *RFC 9019*, 2021.
- [20] Y. He, Z. Zou, K. Sun, Z. Liu, K. Xu, Q. Wang, C. Shen, Z. Wang, and Q. Li, “{RapidPatch}: firmware hot-patching for {Real-Time} embedded devices,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 2225–2242.
- [21] C. Niesler, S. Surminski, and L. Davi, “Hera: Hotpatching of embedded real-time applications,” in *NDSS*, 2021.
- [22] L. Catuogno and C. Galdi, “Secure firmware update: Challenges and solutions,” *Cryptography*, vol. 7, no. 2, p. 30, 2023.
- [23] C. W. Autry and L. Michelle Bobbitt, “Supply chain security orientation: conceptual development and a proposed framework,” *The International Journal of Logistics Management*, vol. 19, no. 1, pp. 42–64, 2008.
- [24] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis, “Sok: Analysis of software supply chain security by establishing secure design properties,” in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, 2022, pp. 15–24.
- [25] W. Enck and L. Williams, “Top five challenges in software supply chain security: Observations from 30 industry and government organizations,” *IEEE Security & Privacy*, vol. 20, no. 2, pp. 96–100, 2022.
- [26] B. M. Reichert and R. R. Obelheiro, “Software supply chain security: a systematic literature review,” *International Journal of Computers and Applications*, vol. 46, no. 10, pp. 853–867, 2024.
- [27] D. Zage, K. Glass, and R. Colbaugh, “Improving supply chain security using big data,” in *2013 IEEE International Conference on Intelligence and Security Informatics*. IEEE, 2013, pp. 254–259.
- [28] K. Park, H. Min, and S. Min, “Inter-relationship among risk taking propensity, supply chain security practices, and supply chain disruption occurrence,” *Journal of Purchasing and Supply Management*, vol. 22, no. 2, pp. 120–130, 2016.
- [29] A. Yeboah-Ofori, S. Islam, S. W. Lee, Z. U. Shamszaman, K. Muhammad, M. Altaf, and M. S. Al-Rakhami, “Cyber threat predictive analytics for improving cyber supply chain security,” *IEEE Access*, vol. 9, pp. 94 318–94 337, 2021.
- [30] B. Hammi and S. Zeadally, “Software supply-chain security: Issues and countermeasures,” *Computer*, vol. 56, no. 7, pp. 54–66, 2023.
- [31] D. Ekwall, “Supply chain security—threats and solutions,” in *Risk management-current issues and challenges*. IntechOpen, 2012.
- [32] A. Bellissimo, J. Burgess, and K. Fu, “Secure software updates: Disappointments and new challenges,” in *HotSec*, 2006.
- [33] O. Alrawi, C. Lever, M. Antonakakis, and F. Monrose, “Sok: Security evaluation of home-based iot deployments,” in *2019 IEEE symposium on security and privacy (sp)*. IEEE, 2019, pp. 1362–1380.
- [34] A. Cui, M. Costello, and S. Stolfo, “When firmware modifications attack: A case study of embedded exploitation,” 2013.
- [35] V. Prakash, S. Xie, and D. Y. Huang, “Inferring software update practices on smart home iot devices through user agent analysis,” in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, 2022, pp. 93–103.
- [36] J. Samuel, N. Mathewson, J. Cappos, and R. Dingleline, “Survivable key compromise in software update systems,” in *Proceedings of the 17th ACM conference on Computer and communications security*, 2010, pp. 61–72.
- [37] R. Tsang, D. Joseph, Q. Wu, S. Salehi, N. Carreon, P. Mohapatra, and H. Homayoun, “Fandemic: Firmware attack construction and deployment on power management integrated circuit and impacts on iot applications,” in *NDSS*, 2022.
- [38] “mitmproxy – an interactive https proxy,” <https://mitmproxy.org/>, 2024.
- [39] “Tapioca – tls certificate inspection tool,” <https://github.com/CERTCC/tapioca>, 2020.
- [40] “Bettercap – the swiss army knife for 802.11, ble, and ethernet networks,” <https://www.bettercap.org/>, 2024.
- [41] “Bdfproxy – backdoor factory proxy,” <https://github.com/secretsquirrel/BDFProxy>, 2019.
- [42] “Process monitor,” <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>, 2024.
- [43] “Ettercap – a comprehensive suite for mitm attacks,” <https://www.ettercap-project.org/>, 2023.